Chapitre 2 : Architecture interne et installation de Kubernetes

Cours : Cloud Computing & DevOps

Pr. Abdelfattah Abassi

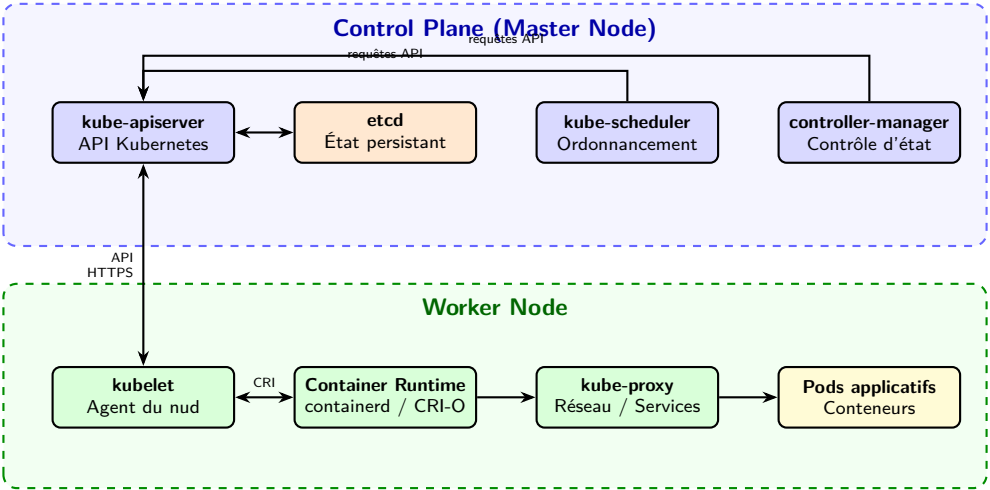
FST Settat – Université Hassan Ier

29 octobre 2026

Plan du chapitre

1. Architecture interne d'un cluster
2. Installation locale sous Windows 11
3. Validation de l'installation

Vue d'ensemble de l'architecture Kubernetes



Les flèches illustrent les flux de communication principaux (API REST, CRI, Routage réseau).

API Server : la porte d'entrée du cluster

Rôle

`kube-apiserver` expose l'API Kubernetes. Les clients et les composants utilisent cette API pour lire ou modifier les ressources.

- Reçoit les requêtes de `kubectl` et des composants.
- Authentifie et autorise les requêtes, puis valide les objets.
- Coordonne l'accès aux données du cluster stockées dans `etcd`.
- Publie les changements que les contrôleurs et autres composants observent.

Exemple

`kubectl get pods` demande à l'API Server la liste des Pods accessibles dans le namespace sélectionné.

etcd, Scheduler et Controller Manager

etcd

- Base clé-valeur.
- Stocke les données persistantes de l'API.
- Ce n'est pas une base métier pour les données des étudiants.

Scheduler

- Repère les Pods non affectés à un nud.
- Évalue les ressources et contraintes.
- Propose un nud approprié.

Controller Manager

- Exécute des boucles de contrôle.
- Compare état réel et état souhaité.
- Déclenche les actions de correction nécessaires.

À ne pas confondre

Le Scheduler choisit un nud pour un Pod en attente. Il ne démarre pas lui-même le conteneur et ne remplace pas le kubelet.

Composants d'un nud : kubelet, runtime et réseau

`kubelet` Agent du nud : reçoit les spécifications des Pods via l'API et veille à ce que leurs conteneurs soient exécutés et surveillés.

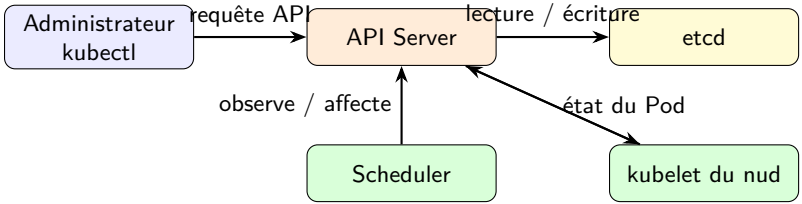
Runtime de conteneurs Télécharge ou utilise les images et exécute les conteneurs via l'interface CRI, par exemple avec `containerd` ou `CRI-O`.

`kube-proxy` Sur les configurations qui l'utilisent, met en place des règles réseau pour acheminer le trafic des Services. Certaines solutions réseau peuvent fournir ces fonctions autrement.

Point essentiel

Kubernetes ne demande pas au kubelet de lancer un conteneur Docker arbitraire : le kubelet travaille avec un runtime compatible CRI.

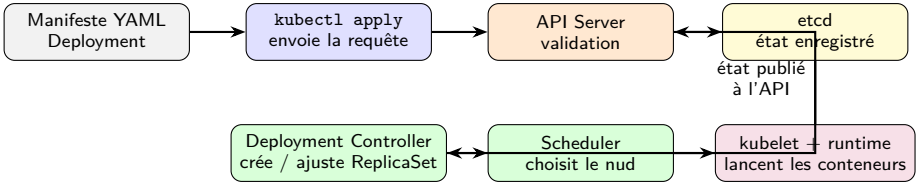
Comment les composants communiquent-ils ?



- L'API Server est le point central d'accès aux ressources.
- Les composants observent l'API et publient leurs décisions ou états.
- Les détails réseau et certains flux internes varient selon l'installation.

Figure 2.2 – Schéma logique, non exhaustif.

Du fichier YAML à l'exécution : `kubectl apply`



1. Le manifeste décrit l'état souhaité ; `apply` l'envoie à l'API.
2. Les contrôleurs créent les ressources nécessaires.
3. Le Scheduler affecte les Pods aux nuds ; le kubelet demande au runtime de les exécuter.

Figure 2.3 – Le déroulement est asynchrone : la commande peut réussir avant que l'application soit prête.

Cycle de création d'un Pod

1. Un utilisateur ou un contrôleur crée un objet Pod via l'API Server.
2. L'API valide la requête et enregistre l'état.
3. Le Scheduler choisit un nud si le Pod n'en possède pas encore.
4. Le kubelet de ce nud observe le Pod affecté.
5. Le runtime récupère l'image si nécessaire et crée les conteneurs.
6. Les sondes et l'état d'exécution permettent de suivre la santé du Pod.
7. Le kubelet publie les informations d'état via l'API.

Attention

Un Pod en état `Running` n'implique pas toujours que l'application est prête à recevoir du trafic : les sondes de disponibilité comptent.

Réconciliation et auto-réparation

Exemple : trois répliques souhaitées

- État souhaité : 3 Pods.
- État réel : 2 Pods disponibles.
- Le contrôleur observe l'écart.
- Kubernetes tente de créer un Pod de remplacement.

Limite

L'auto-réparation n'empêche pas toutes les pannes. Elle dépend de l'état du cluster, des ressources disponibles, des volumes, du réseau et de la configuration applicative.

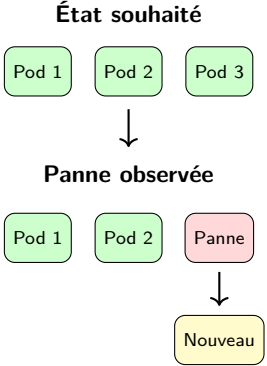


Figure 2.4 – Illustration simplifiée du remplacement d'une réplique.

Choix de l'environnement de TP

Méthode principale

Docker Desktop avec son cluster Kubernetes intégré, lorsque cette fonctionnalité est disponible dans la version installée.

- Installation et gestion relativement simples pour un TP local.
- Le cluster s'exécute sur la machine de l'étudiant.
- Docker Desktop peut proposer un cluster à un nud (kubeadm) ou un cluster multi-nuds local (kind), selon les options disponibles.
- Les libellés exacts de l'interface dépendent de la version.

Important

Un cluster local sert à apprendre et tester. Il ne remplace pas une infrastructure de production sécurisée et hautement disponible.

Prérequis avant l'installation

- Windows 11 pris en charge par la version actuelle de Docker Desktop.
- Virtualisation matérielle activée dans le BIOS/UEFI.
- WSL 2 opérationnel si Docker Desktop utilise le moteur WSL 2.
- Docker Desktop installé et démarré.
- Mémoire, processeur et espace disque suffisants pour Windows, Docker Desktop et les composants Kubernetes.
- Connexion Internet disponible pour télécharger les images requises.
- Droits d'installation et politique de sécurité de l'établissement compatibles.

Vérification initiale

Commencez par ouvrir Docker Desktop et attendez que son moteur soit opérationnel avant de diagnostiquer Kubernetes.

Vérifier Docker depuis PowerShell

Ouvrir PowerShell ou le terminal intégré de VS Code, puis exécuter :

```
docker version
```

- Affiche les informations du client Docker et, si accessible, du serveur.
- Si seule la partie Client apparaît ou si une erreur de connexion survient, le moteur Docker peut ne pas être démarré ou accessible.

```
docker info
```

- Donne des informations détaillées sur le moteur : système, ressources, stockage, runtime et configuration.
- Les champs affichés varient selon la version et la configuration.

À ne pas faire

Ne recopiez pas une sortie supposée : comparez les résultats réels de votre machine.

Activer Kubernetes dans Docker Desktop

1. Démarrer Docker Desktop et attendre que le moteur soit prêt.
2. Ouvrir la vue Kubernetes ou les paramètres Kubernetes proposés par la version installée.
3. Si l'interface propose la création d'un cluster, choisir le type souhaité et lancer sa création.
4. Si l'option d'activation est disponible, activer Kubernetes et appliquer la configuration demandée.
5. Attendre la fin du téléchargement des images et du démarrage des composants.
6. Ne pas lancer les tests de cluster tant que le démarrage n'est pas terminé.

Variations de l'interface

Selon la version, l'interface peut présenter un bouton de création de cluster, un réglage d'activation ou des options de provisioning. Suivez les libellés réellement affichés ; ne supposez pas qu'ils sont identiques sur toutes les versions.

Vérifier kubectl et le contexte actif

```
kubectl version --client
```

Affiche la version du client `kubectl`. Elle doit rester compatible avec la version du serveur Kubernetes (écart maximal recommandé : une version mineure).

```
kubectl config current-context
```

Affiche le contexte sélectionné. Le nom du contexte dépend de l'installation : ne supposez pas qu'il est identique chez tous les étudiants.

Contexte

Un contexte associe notamment un cluster, un utilisateur et éventuellement un namespace. Vérifiez-le avant d'exécuter des commandes qui modifient des ressources.

Vérifier le cluster Kubernetes

```
kubectl cluster-info
```

Affiche les informations de connexion aux services principaux du cluster lorsque celui-ci est joignable.

```
kubectl get nodes
```

Liste les nuds connus par l'API. Pour un cluster local simple, un seul nud peut être normal ; en mode multi-nuds, plusieurs peuvent apparaître.

```
kubectl get pods -A
```

Liste les Pods de tous les namespaces. Certains Pods système peuvent être encore en démarrage juste après la création du cluster.

Problèmes fréquents et diagnostic

Symptôme	Pistes de vérification
`docker version` ne joint pas le serveur	Vérifier que Docker Desktop est démarré et que son moteur est prêt.
`kubectl` n'est pas reconnu	Vérifier l'installation, le `PATH` et rouvrir le terminal après modification.
`cluster-info` refuse la connexion	Vérifier que Kubernetes est démarré et que le contexte pointe vers le bon cluster.
Nud non `Ready`	Attendre le démarrage, puis examiner les composants système et les événements.
Pods système en attente	Vérifier les téléchargements d'images, l'accès réseau, les ressources et les événements.
Mauvais cluster sélectionné	Exécuter `kubectl config current-context` avant toute opération.

Docker Desktop, Minikube et kind

Outil	Intérêt pédagogique	À savoir
Docker Desktop Kubernetes	Intégration pratique avec Docker Desktop	Fonctionnalité et options disponibles selon la version ; cluster local.
Minikube	Cluster local pour apprendre Kubernetes	Utilise un pilote compatible, par exemple Docker ou une VM ; démarrage via `minikube start`.
kind	Crée des clusters Kubernetes à partir de conteneurs	Utile pour tester des clusters multi-nuds localement ; les nuds sont des conteneurs.

Conseil pour le TP

Choisir une seule méthode principale pour le groupe. N'installez pas simultanément plusieurs clusters sans raison : les contextes multiples peuvent créer de la confusion.

Cluster local et cluster de production

Cluster local

- Fonctionne sur un poste de travail.
- Destiné aux exercices et tests.
- Ressources et disponibilité limitées par le poste.
- Peut être réinitialisé facilement.

Production

- Infrastructure conçue pour les applications réelles.
- Contrôle d'accès et secrets gérés rigoureusement.
- Sauvegardes, supervision, mises à jour et plan de reprise.
- Capacité, sécurité et haute disponibilité planifiées.

Conclusion

Le succès d'un TP local prouve que l'environnement d'apprentissage fonctionne ; il ne prouve pas qu'une architecture est prête pour la production.

Checklist de validation

- ☐ Docker Desktop est installé et son moteur est démarré.
- ☐ `docker version` affiche le client et le serveur.
- ☐ `docker info` renvoie les informations du moteur.
- ☐ `kubectl version -client` fonctionne.
- ☐ `kubectl config current-context` affiche le contexte prévu.
- ☐ `kubectl cluster-info` contacte le cluster.
- ☐ `kubectl get nodes` affiche le ou les nuds.
- ☐ `kubectl get pods -A` permet d'observer les Pods système.
- ☐ Les problèmes éventuels ont été décrits et diagnostiqués.

Cochez les éléments uniquement après vérification sur votre machine.

Exercice de validation de l'installation

Travail individuel – 20 à 30 minutes

1. Ouvrir Docker Desktop et vérifier que Kubernetes est opérationnel.
2. Exécuter les sept commandes de vérification étudiées.
3. Relever le contexte actif et le nombre de nuds.
4. Exécuter :

```
kubectl get pods -A
```

5. Choisir un Pod système et utiliser `kubectl describe pod` avec son nom et son namespace.
6. Rédiger un court compte rendu : commandes, observations réelles, éventuelles erreurs et corrections effectuées.

Critères de réussite

Le client Docker répond, le contexte est identifié, l'API Kubernetes est joignable, les nuds sont inspectables et l'étudiant sait expliquer une observation sans inventer de sortie.

Ressources officielles

- Kubernetes – composants du cluster : `https://kubernetes.io/docs/concepts/overview/components/`
- Kubernetes – installation de kubectl sous Windows : `https://kubernetes.io/docs/tasks/tools/install-kubectl-windows/`
- Docker Desktop – vue Kubernetes : `https://docs.docker.com/desktop/use-desktop/kubernetes/`
- Docker Desktop – paramètres : `https://docs.docker.com/desktop/settings-and-maintenance/settings/`
- Minikube – démarrage : `https://minikube.sigs.k8s.io/docs/start/`

Pour le prochain chapitre

Une fois le cluster vérifié, nous pourrons déployer une première application avec un manifeste YAML, observer ses Pods et l'exposer à l'aide d'un Service.